A Retail Digital Rupiah Prototype on Hyperledger Fabric: Selected Controls and a Single-Host Performance Baseline

1st Muhammad Fachrizal Giffari
Department of Electrical and Information Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
muhammadfachrizalgiffari@mail.ugm.ac.id

2nd Teguh Bharata Adji
Department of Electrical and Information Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
adji@ugm.ac.id

3rd Guntur Dharma Putra
Department of Electrical and Information Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
gdputra@ugm.ac.id

***Abstract*—Bank Indonesia's Project Garuda publicly documents wholesale central bank digital currency. This paper reports a retail Digital Rupiah prototype on Hyperledger Fabric 2.5 and a compliance study of its customer controls and endorsement boundaries. The study uses four peer MSPs: CentralBankMSP, DistributorMSP, EndUserMSP, and the Observer (MSP ID: ValidatorMSP). Three Go chaincodes cover monetary state, customer controls, and payments. The Observer has read-only audit access through an Auditor role. Server-managed X.509 identities distinguish the signing actor, customer subject, and wallet custodian.**

In run-20261014-01, reported under the earlier identifiers, the correctness record shows 20 of 20 synthetic scenarios, 207 of 207 authorization cells, 27 of 27 validation cases, and 15 of 15 action outcomes; the post-rename correctness re-run is pending. These checks cover only the selected scenarios; they do not establish identity authenticity, screening effectiveness, actual PPATK reporting, or institutional compliance.

The compliance study also reports throughput and latency from a one-worker Hyperledger Caliper run on a single host. Findings are limited to synthetic functional tests and single-host performance measurements.

***Index Terms*—Digital Rupiah, retail CBDC, KYC and AML/CFT controls, endorsement, Hyperledger Fabric, two-tier distribution, performance baseline**

I. Introduction

Central banks are evaluating Central Bank Digital Currency (CBDC) as a digital form of sovereign money [1]. Bank Indonesia pursues CBDC through Project Garuda. Its publicly documented proof of concept targets the *wholesale* tier, specifically a cash ledger for issuance and redemption among the central bank and large financial institutions [2], [3]. The Garuda White Paper also describes a retail tier in which intermediary banks distribute Digital Rupiah to households and merchants [3]. That description remains at the policy and architecture level, and we are not aware of a publicly documented, testable system that implements Bank Indonesia's retail money business processes end to end on a blockchain ledger.

Central-bank digital money involves a sequence of business processes, not a single ledger operation. The central bank *issues* wholesale Digital Rupiah (w-Rupiah Digital) directly into a Distributor reserve held by a wholesale or retail bank. A retail bank *converts* reserve balances into retail Digital Rupiah (r-Rupiah Digital) held as a retail float, and funds customer wallets from that float. The retail bank is the only Custodian and serves customers and merchants in the retail tier. Holders *transact*, redemption retires w-Rupiah Digital from a Distributor reserve, and an Observer institution, acting through a read-only Auditor role, audits the circulation. A blockchain implementation must express these processes as deterministic transactions that each organization can validate independently. It must also preserve two-tier intermediation to reduce bank disintermediation [4] and give the Observer an audit surface.

Most technical studies isolate one part of this problem: permissioned CBDC architecture [5], privacy [6], security threat models [7], or throughput in account-based systems [8]. Deployed retail systems, including Nigeria's eNaira [9], China's e-CNY [10], and the Bahamian Sand Dollar [11], publish design papers but not reproducible performance benchmarks. The surveyed literature therefore provides no integrated, measurable retail reference for the Indonesian two-tier model [12], [13].

The compliance study is the main study of this paper. It tests whether selected customer controls and organizational endorsement boundaries behave as designed in synthetic scenarios. It also measures the throughput and latency of the same three chaincodes with a Hyperledger Caliper benchmark (Section VI).

To address this gap, we implement a Digital Rupiah prototype on a permissioned Hyperledger Fabric network [14]. The prototype maps Bank Indonesia's business processes to chaincode transactions. This paper contributes:

1) The compliance study of the prototype on a permissioned network with four peer MSPs and three domain chaincodes for monetary operations, customer controls, and payments. Synthetic integration scenarios exercise distinct senior approval, restriction evidence, reserve

conversion and wallet funding, read-only validation, and the combined endorsement requirements.

2) A two-tier Digital Rupiah implementation with hierarchical issuance and distribution, retail transfers, and read-only auditor access to subject records. This extends the wholesale-only Project Garuda proof of concept with a testable retail implementation.
3) A deterministic transfer protocol with explicit eligibility, limit, and balance checks, idempotent receipts keyed by business reference, Fabric multi-version concurrency control (MVCC), and atomic wallet updates. The protocol lets endorsing organizations apply the same state-transition rules independently.
4) A Hyperledger Caliper [15] benchmark of the compliance study. It documents the testbed, the 31 workloads, and the functional checks for authorization, validation, and action outcomes. It reports throughput and latency under a single-host, one-worker scope.

II. Background and Related Work

A. Two-Tier Retail CBDC

In the two-tier (intermediated) model the central bank issues CBDC and remains the ultimate liability holder, while regulated intermediaries serve end users and operate customer-facing rails [1], [4]. This preserves the existing division of labor and limits the central bank's exposure to retail operations. Bank Indonesia adopts this structure for Digital Rupiah, with Distributor Banks acting as distributors between the central bank and the public [3].

B. Permissioned Blockchain and Hyperledger Fabric

Hyperledger Fabric is a modular, permissioned blockchain with pluggable ordering, channel-based data isolation, and separate execution, ordering, and validation stages [14]. Membership Service Providers (MSPs) govern identity, the crash-fault-tolerant Raft protocol orders transactions, and CouchDB can provide a queryable world state. Fabric has consequently been used in CBDC and regulated-finance prototypes [5], [16]. Hyperledger Caliper [15], [17] measures throughput and latency under different workloads.

C. Authorization and Oversight

Retail CBDC systems require explicit authorization boundaries, safe balance updates, and read-only audit visibility. Token-based retail designs also face denial-of-service and double-spend risks [18], so transfer validation must be deterministic. Our chaincode combines organization-aware authorization, subject eligibility and limit checks, and read-only Observer access to subject records. This complements studies that address privacy [6] or security [7] separately. The closest published prototype is Project Aurum, a full-stack two-tier retail CBDC built for the Hong Kong Monetary Authority [19], which pairs a wholesale interbank system with a retail e-wallet system and, like this work, keeps the two tiers architecturally distinct. Aurum reports no reproducible throughput benchmark, and its source code and technical manuals are released only to BIS member central banks rather than publicly. What distinguishes this prototype among the representative systems in Table I is that its two-tier intermediation and its authorization and oversight controls are realized in an implementation with a measured throughput and latency baseline, rather than described in design documentation alone.

TABLE I
Comparison with Representative Retail CBDC Systems

System	Platform	2-tier	Open	Bench.
e-CNY [10]	Permissioned	✓	×	×
eNaira [9]	Fabric	✓	×	×
Sand Dollar [11]	Permissioned	✓	×	×
OpenCBDC [8]	Account	×	✓	✓
This work	Fabric 2.5	✓	✓	✓

Open: implementation publicly available. Bench.: throughput and latency benchmark reported. OpenCBDC is account-based and not distributed-ledger-based. Rows span deployed national CBDCs and a research platform, and summarize public information rather than a controlled comparison.

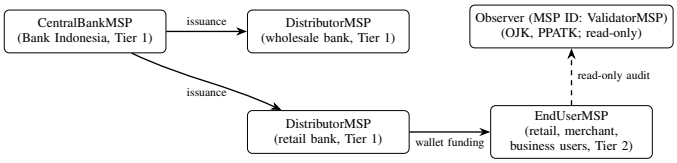


Fig. 1. Two-tier actor structure of the Digital Rupiah prototype, mapped to the four Fabric MSPs. CentralBankMSP (Bank Indonesia) issues w-Rupiah Digital directly into the Distributor reserves of the wholesale and retail banks (DistributorMSP). The retail bank converts reserve balances into r-Rupiah Digital and funds retail, merchant, and business wallets (EndUserMSP) from its retail float. The Observer (MSP ID: ValidatorMSP, for OJK and PPATK) holds read-only audit visibility, shown as a dashed flow.

III. Bank Indonesia Business Processes

The system implements seven processes in the circulation of central-bank digital money. Fig. 1 shows the actors in each tier, and Table II maps each process to the chaincode and transactions that realize it.

The institutional tier contains CentralBankMSP (Bank Indonesia) and the Distributor Banks, which are wholesale or retail banks (DistributorMSP). Bank Indonesia registers each Distributor Bank with its kind through `RegisterDistributor` and issues w-Rupiah Digital with `Issue`, which credits the bank's Distributor reserve and the wholesale supply. Banks move w-Rupiah Digital between reserves through `TransferReserve`. Bank Indonesia retires it through `RedeemReserve`. A retail bank converts reserve balances into its retail float through `ConvertToRetail` and funds customer wallets from that float through `FundWallet`. Customers and merchants (EndUserMSP) then transfer value through `Transfer`, which checks subject eligibility, limits, and balances before the state changes. Customer subjects move through pending creation, review, a standard or high-risk decision, restriction, and release on approval.

IV. System Architecture

Fig. 2 shows the four-layer architecture: client, backend API, off-chain store, and Fabric network. The web client calls

TABLE II
BUSINESS PROCESSES AND THEIR CHAINCODE REALIZATION

Process	Chaincode	Transactions
Issuance	rupiah-monetary	`RegisterDistributor`, `Issue`
Interbank transfer	rupiah-monetary	`TransferReserve`
Redemption	rupiah-monetary	`RedeemReserve`
Conversion	rupiah-payments	`ConvertToRetail`
Wallet funding and payment	rupiah-payments	`OpenWallet`, `FundWallet`, `Transfer`
Customer controls	rupiah-controls	`CreatePending`, `ProtectSubject`, `RecordReview`, `ApproveStandard`, `DecideHighRisk`, `SetLimits`, `SetBusinessAuthority`
Restriction and staff controls	rupiah-controls	`FreezeSubject`, `RequestUnfreeze`, `ApproveUnfreeze`, `CreateStaffPending`, `ProtectStaff`, `SetStaffRole`

The Observer (ValidatorMSP) reads subject records through `GetSubject` and is not an ordinary endorser of these transactions.

a Go REST backend built with Gorilla Mux. The backend authenticates requests with JWT and role-based access control, stores account and authentication data in PostgreSQL, and submits ledger transactions through the Fabric Gateway SDK. Fig. 3 details the permissioned network.

A. Network Topology

The four-organization network runs one peer per organization on one application channel, `bi-coin` (Fig. 3, Table III). The measured run used the earlier channel name `rupiah-study`, as Section VI discloses. `CentralBankMSP` represents Bank Indonesia, the issuer. It also operates the single orderer and hosts its certificate authority `ca.centralbank`. `DistributorMSP` represents the wholesale and retail Distributor Banks that hold Distributor reserves. `ValidatorMSP` is the Observer for the Financial Services Authority (OJK, *Otoritas Jasa Keuangan*) and the Indonesian Financial Transaction Reports and Analysis Center (PPATK, *Pusat Pelaporan dan Analisis Transaksi Keuangan*). It has read-only audit access through an Auditor role and is not an ordinary payment endorser of the three chaincodes. `EndUserMSP` represents retail, merchant, and business users. Each organization operates one peer with its own CouchDB world state and MSP, and the current network gives each organization its own Fabric CA (Section VI notes the CA setup of the measured run). A single Raft orderer orders transactions. This configuration provides ordering but not crash fault tolerance, which requires at least three consenters. The channel's `Endorsement` and `LifecycleEndorsement` policies require a majority of the four organizations. ValidatorMSP approves chaincode definitions under this lifecycle policy, but every chaincode endorsement policy in Table IV excludes it.

TABLE III
TOPOLOGY OF THE FOUR-ORGANIZATION FABRIC NETWORK

Organization	Role	Peer	Host port
CentralBankMSP	Issuer; hosts orderer and CA	peer0.centralbank	17051
DistributorMSP	Wholesale and retail Distributor Banks	peer0.distributor	18051
ValidatorMSP	Observer; read-only audit (OJK, PPATK)	peer0.validator	19051
EndUserMSP	End users (retail, merchant, business)	peer0.enduser	20051

Full peer names end in `.bi-coin`, for example `peer0.centralbank.bi-coin`. Peers listen on container port 7051. Orderer: `orderer.centralbank.bi-coin` (single-node Raft, host port 17050). Certificate authorities: `ca.centralbank`, `ca.distributor`, `ca.validator`, and `ca.enduser`, one per organization. One CouchDB per organization.

B. Compliance Study Design

The compliance study runs on an isolated Fabric 2.5 network with one channel, `bi-coin`, and four peer MSPs: `CentralBankMSP`, `DistributorMSP`, `ValidatorMSP`, and `EndUserMSP`. CentralBankMSP also operates the single orderer. ValidatorMSP, the Observer, provides read-only audit access and is not an ordinary payment endorser. The study uses a separate PostgreSQL volume and synthetic state. Server-managed X.509 signers are mapped per application username; the signing identity, customer subject, and wallet custodian are represented separately. Fig. 4 summarizes this topology and the three chaincode policies. Section VI reports the measured capacity and latency.

The implementation separates monetary state, customer controls, and payments into three chaincodes. Table IV shows their default policies and state ownership.

Effective-control keys use a state-based policy requiring CentralBankMSP and DistributorMSP. `ConvertToRetail` invokes the monetary and payments chaincodes, and `FundWallet` invokes the controls and payments chaincodes, each in one same-channel transaction, so the endorsement policies of both chaincodes apply to each flow. These policies and distinct-staff approvals are experimental design choices; they are not presented as statutory endorsement rules.

C. On-Chain and Off-Chain Data

The three Go chaincodes group their functions by domain (Table V). The monetary chaincode holds wholesale and retail supply and distributor reserves. The controls chaincode holds pseudonymous subjects, KYC and risk status, business authority, limits, and restriction evidence. The payments chaincode holds wallets, retail floats, payment receipts, and usage counters. Account and authentication data remain in the off-chain PostgreSQL store, while the ledger contains the wallet and transaction state needed for settlement and oversight.

A Go backend enforces role-based access control over seven role groups, summarized in Table VI, before any transaction reaches the chaincode.

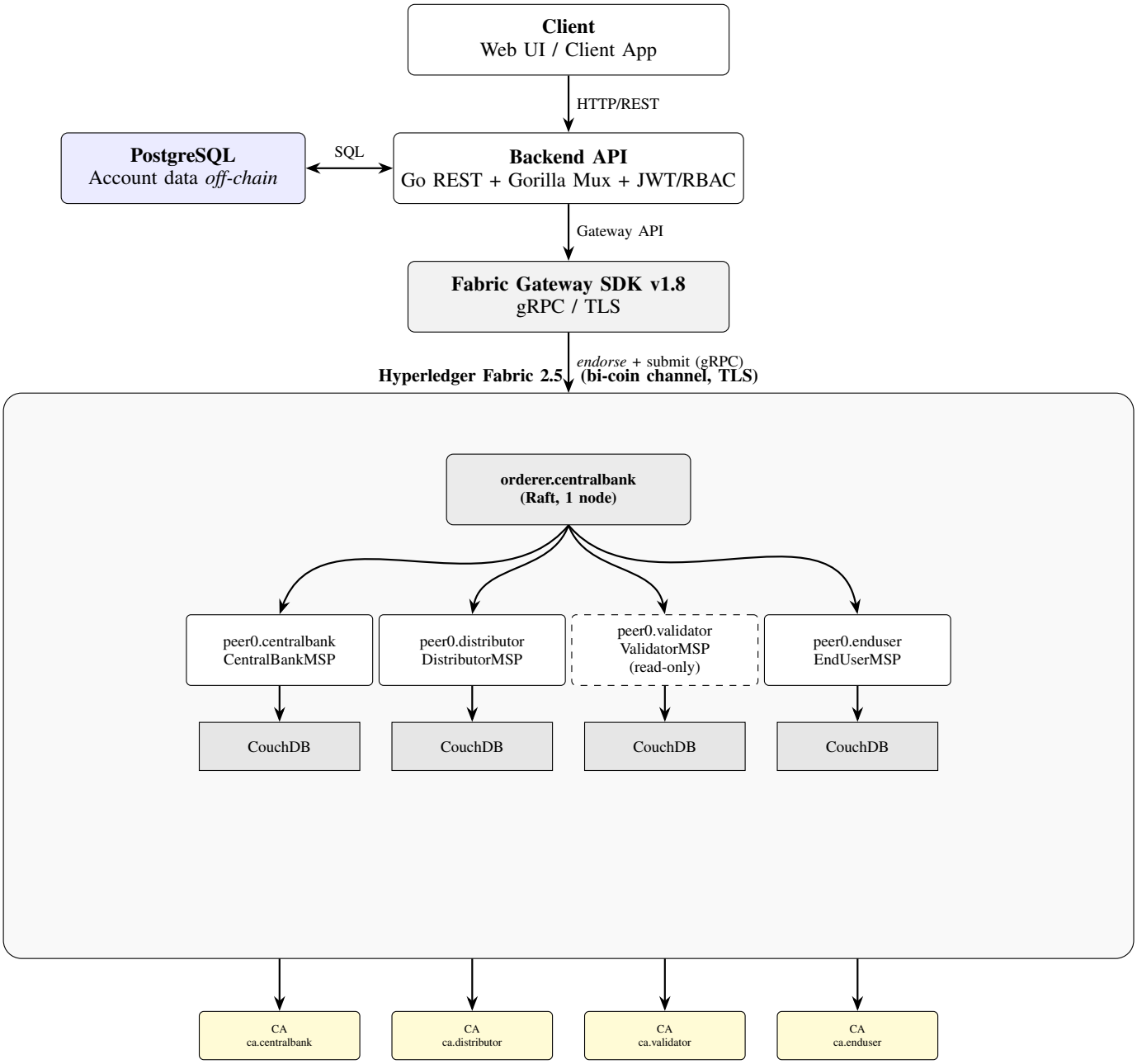


Fig. 2. Architecture of the Digital Rupiah prototype: client layer, Go REST backend with JWT/RBAC, off-chain PostgreSQL store, Fabric Gateway SDK, and the Hyperledger Fabric 2.5 network of four peer organizations on the bi-coin channel, with one CouchDB and one certificate authority per organization and a single-node Raft orderer.

TABLE IV
COMPLIANCE STUDY CHAINCODE DOMAINS AND ENDORSEMENT POLICIES

Chaincode	State owned	Default endorsement
`rupiah-monetary`	Wholesale and retail supply, distributor reserves	`AND('CentralBankMSP.peer','DistributorMSP.peer')` `AND('DistributorMSP.peer',`
`rupiah-controls`	Pseudonymous subjects, KYC/risk status, business authority, limits, restriction evidence	`OR('CentralBankMSP.peer','EndUserMSP.peer'))`
`rupiah-payments`	Wallets, retail floats, payment receipts, usage limits	`AND('DistributorMSP.peer','EndUserMSP.peer')`

The two layers enforce different things, and the split matters for how the prototype's guarantees should be read. The backend maps JWT roles to permitted operations and restricts customers to their own wallets. The chaincode checks the caller's

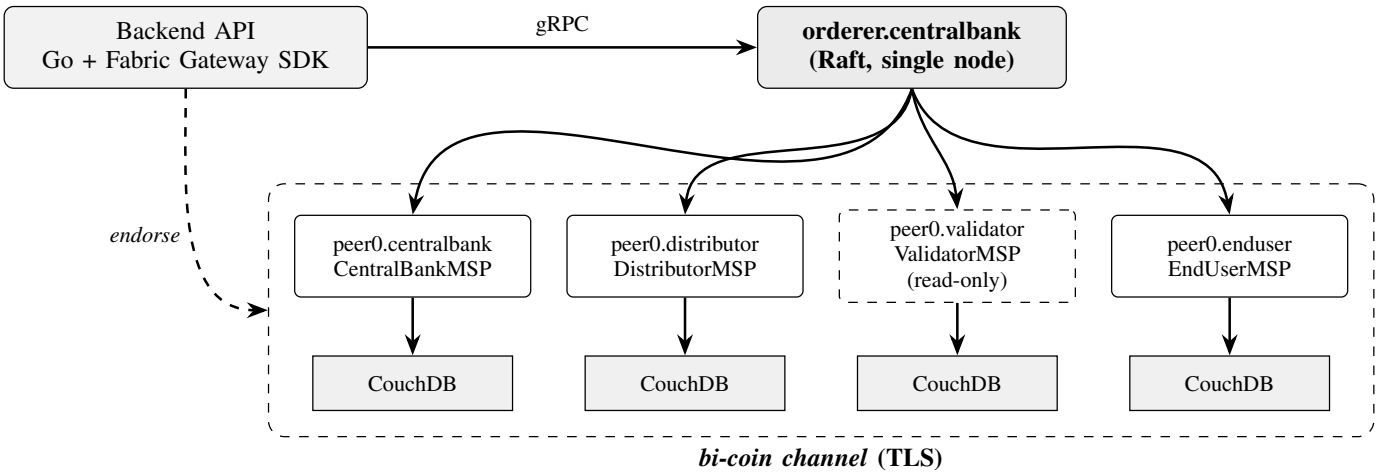


Fig. 3. Hyperledger Fabric topology of the four-organization network: one peer per organization, each with a CouchDB world state, on the bi-coin channel under a single-node Raft orderer. The Go backend endorses through the Fabric Gateway SDK.

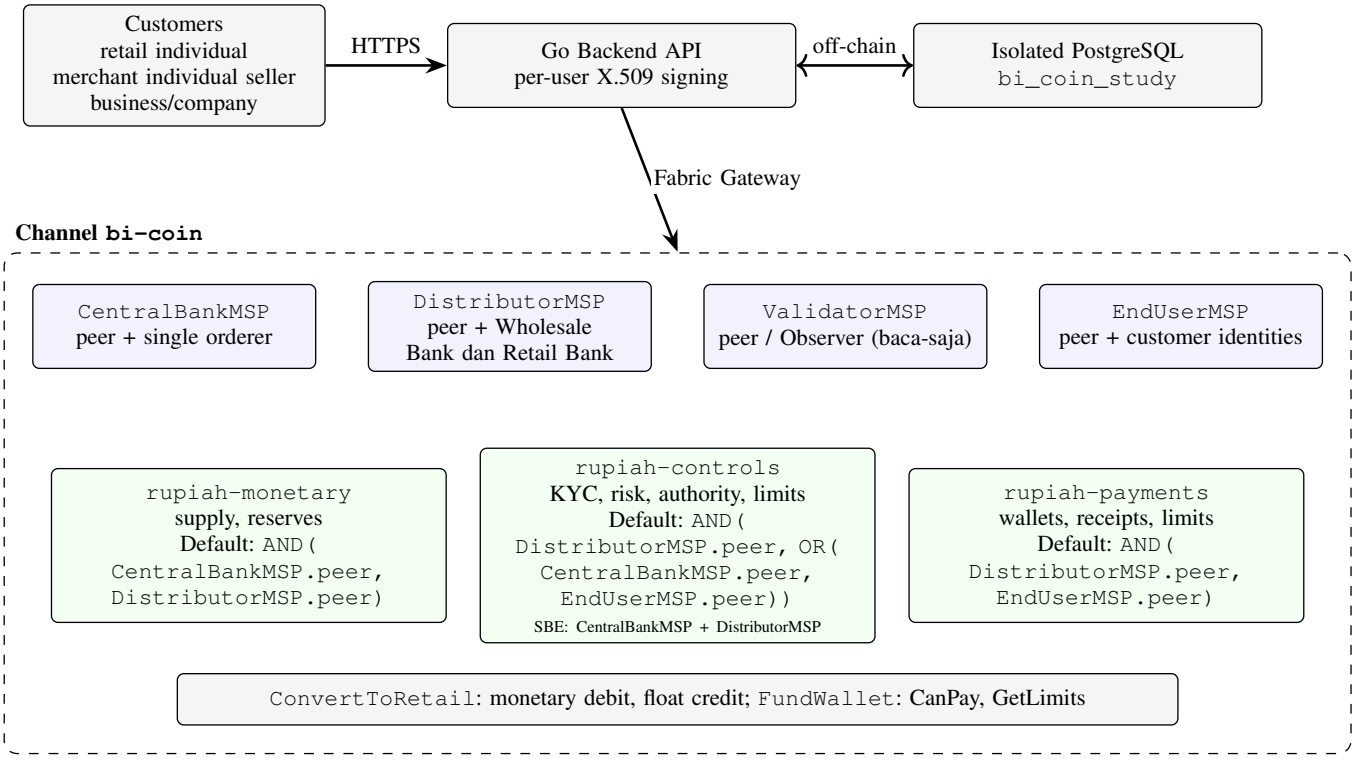


Fig. 4. Four-MSP compliance study architecture, chaincode domains, and endorsement boundaries.

MSP, subject eligibility, limits, and available balance before it writes the state transition. Every endorsing peer re-evaluates these rules independently. The single backend and PostgreSQL instance remain trust and availability concentration points for this prototype.

V. Protocol and Determinism

The payments chaincode applies explicit eligibility, limit, and balance checks during endorsement. We formalize these checks and examine their behavior under concurrent transactions.

A. Transfer-Validity Rules

For a transfer of amount a from subject s to subject r, the payments chaincode admits the transfer only if every one of the following checks passes:

- the caller is an EndUserMSP identity that can act for s, as evaluated by the controls chaincode (`CanActFor`);
- s and r are both active and unrestricted (`CanPay`);

TABLE V
CHAINCODE FUNCTION GROUPS

Chaincode	Functions
rupiah-monetary	`Initialize`, `RegisterDistributor`, `Issue`, `TransferReserve`, `RedeemReserve`, `DebitReserveForConversion`, `GetWholesaleSupply`, `GetRetailSupply`, `GetTotalSupply`, `GetReserve`
rupiah-controls	`GetCallerIdentity`, `CreatePending`, `CreateStaffPending`, `ProtectSubject`, `ProtectStaff`, `SetStaffRole`, `RecordReview`, `ApproveStandard`, `DecideHighRisk`, `SetLimits`, `SetBusinessAuthority`, `FreezeSubject`, `RequestUnfreeze`, `ApproveUnfreeze`, `CanPay`, `CanActFor`, `GetLimits`, `GetSubject`
rupiah-payments	`OpenWallet`, `Transfer`, `ConvertToRetail`, `FundWallet`, `GetRetailFloat`, `GetBalance`, `GetReceipt`

TABLE VI
BACKEND ROLES AND REPRESENTATIVE PERMISSIONS

Role	Representative permitted actions
Public	Health check, login
Authenticated	Read own identity; submit a subject for onboarding
Customer (KYC-verified or merchant, EndUserMSP)	Open wallet, retail transfer, read own balance
Wholesale bank (DistributorMSP)	Interbank transfer
Retail bank (DistributorMSP)	Interbank transfer; conversion to retail float; wallet funding; review, limit, decide on, and restrict subjects
Bank Indonesia (CentralBankMSP)	Staff administration, monetary initialization, register distributors, issue, redeem reserve
Auditor (ValidatorMSP)	Read subject records (read-only)

- $a > 0$, $s \neq r$, and a non-empty reference are present;
- a does not exceed the per-transaction limit of s, and the daily and monthly payment totals of s are checked against its limits; and
- s has enough balance to cover a, and the balance of r after receipt does not exceed its holdings limit.

If any check fails, the transfer is rejected. A successful transfer decreases the sender balance, increases the receiver balance, and writes a receipt keyed by the reference. A repeated call with the same reference and parameters returns the stored receipt. A reference reused for a different operation is rejected. Issuance and distributor registration require CentralBankMSP. Each bank identity carries its distributor identifier in a `distributor_id` certificate attribute, and the chaincode rejects any debit of another bank's reserve or float. Conversion is limited to retail banks, so only they hold a retail float.

1: **function** TRANSFER(sender, receiver, amount, reference)
2: **if** caller is not EndUserMSP **or** caller cannot act for sender **then**
3: **return** REJECT
4: **end if**
5: **if** sender or receiver is inactive or restricted **then**
6: **return** REJECT
7: **end if**
8: **if** amount ≤ 0 **or** sender = receiver **or** reference is missing **then**
9: **return** REJECT
10: **end if**
11: **if** reference has a stored receipt **then**
12: **return** stored receipt if it matches, otherwise REJECT
13: **end if**
14: **if** amount exceeds the per-transaction, daily, or monthly limit **then**
15: **return** REJECT
16: **end if**
17: **if** sender.balance $<$ amount **or** receiver.balance + amount exceeds holdings limit **then**
18: **return** REJECT
19: **end if**
20: sender.balance $\leftarrow$ sender.balance $-$ amount
21: receiver.balance $\leftarrow$ receiver.balance $+$ amount
22: **PutState** both wallets and the receipt (validated by Fabric MVCC)
23: **return** COMMIT

Fig. 5. Authorized, atomic, and double-spend-safe transfer executed by the payments chaincode. Fabric MVCC validates the wallet versions before commit.

B. Determinism and Double-Spend Safety

Every endorsing peer evaluates these checks against the same transaction arguments and ledger state. The chaincodes do not use a peer-local clock or random values, and usage windows are derived from the transaction timestamp. Each successful transfer updates the sender and receiver records and writes the receipt within one transaction, so every endorser evaluates the same balance transition before validation.

Double-spend safety follows from Fabric's MVCC. An endorsement reads the current version of each wallet record, and Fabric commits the transaction only if those versions remain unchanged through ordering and validation. Two concurrent transfers from the same wallet may endorse against the same version, but only the first ordered transaction commits. The second fails MVCC validation after the first update. Because the wallet records are keys in the read set, one version check covers the balances used by the transfer and prevents a stale balance update.

VI. PERFORMANCE EVALUATION

A. Methodology

The compliance benchmark runs Hyperledger Caliper 0.6.0 with one worker on one host, against the four-organization

TABLE VII
TESTBED CONFIGURATION OF THE COMPLIANCE BENCHMARK

Component	Specification
Host	One host; Intel Core i7-7700HQ, 4 cores, 8 threads, 2.80 GHz
Memory	15.52 GiB
Isolation	Docker-in-Docker
Fabric	2.5 images; 2.5.16 tools bundle
CouchDB	3.3, one instance per organization
Fabric CA	1.5.9; one CA (`ca.centralbank`) in the measured run
Caliper	0.6.0; one worker; 30 s confirmation windows
Chaincodes	`rupiah-monetary`, `rupiah-controls`, `rupiah-payments`
Channel	`bi-coin` (`rupiah-study` in the measured run)
Organizations	CentralBankMSP, DistributorMSP, ValidatorMSP, EndUserMSP; one peer each
Ordering	Single-node Raft; batch timeout 2 s; maximum 10 messages

Figures come from run-20261014-01 under the earlier identifiers (channel `rupiah-study`, one Fabric CA, `ca.centralbank`). Afterwards, identifiers renamed to `bi-coin`; configuration and endorsement policies unchanged; identities reissued under per-org Fabric CAs; correctness re-run pending.

network and its three chaincodes. Each workload is confirmed over three 30-second rounds, and its verified throughput is the mean of the three rounds. A probe exceeds the drain limit when verified throughput over the probe duration plus drain falls below 90 percent of the delivered rate. The testbed has four peer organizations, one peer per organization, a single-node Raft orderer, and one CouchDB instance per organization.

B. Results

In run-20261014-01 under the earlier identifiers, all 31 workloads passed their preflight checks. The correctness record of that run shows 20 of 20 scenarios, 207 of 207 authorization cells, 27 of 27 validation cases, and 15 of 15 action outcomes. The post-rename correctness re-run is pending.

Thirty of 31 workloads are bracketed by measured probes. The fund-reserve upper bound is unresolved because two probes failed on fetch errors before measurement. Its nearest measured failure, 0.21875 ops/s, lies above the confirmed 0.183594 ops/s. Confirmed write rates range from 0.183594 to 4.75 TPS across the 21 write workloads, and 19 of 21 confirm at 0.78125 TPS or below (Table VIII). Read workloads confirm at 272 to 296 ops/s. Under the earlier identifiers, every read confirmation round in run-20261014-01 passed. The two payments reads confirm at 272 ops/s, below the 296 ops/s of the controls and monetary reads. In failing read probes, delivered rate stayed below the offered target, so the read rates are lower bounds on read capacity.

For every write workload except fund-reserve, the first failing probe completed transactions after its probe window and recorded no failed or timed-out transactions. Verified throughput over the probe duration plus drain therefore fell below 90 percent of the delivered rate. The configured two-second batch timeout is a plausible source of this drain latency, but the run records no block contents, so the link is not isolated.

TABLE VIII
CONFIRMED CAPACITY AND NEAREST MEASURED FAILURE PER WORKLOAD (WRITES IN TPS, READS IN OPS/S)

Workload	Confirmed	Nearest failing
rupiah-monetary		
register-distributor	0.78125	0.828125
issue	0.195313	0.207031
interbank-transfer	TBD	TBD
redeem-reserve	0.195313	0.207031
read-supply	296	320
read-reserve	296	320
rupiah-controls		
create-pending	4.75	5.125
protect-subject	0.78125	0.828125
create-staff-pending	0.195313	0.207031
protect-staff	0.78125	0.828125
set-staff-role	0.78125	0.828125
record-review	0.78125	0.828125
approve-standard	0.78125	0.828125
decide-high-risk	0.195313	0.207031
set-limits	0.78125	0.828125
set-business-authority	0.78125	0.828125
freeze-subject	0.78125	0.828125
request-unfreeze	0.78125	0.828125
approve-unfreeze	0.195313	0.207031
read-caller-identity	296	320
can-pay	296	320
read-limits	296	320
can-act-for	296	320
validator-read-subject	296	320
rupiah-payments		
open-wallet	4.625	5
transfer-retail	0.78125	0.828125
transfer-merchant	0.195313	0.207031
convert-to-retail	TBD	TBD
fund-wallet	TBD	TBD
read-balance	272	296
read-receipt	272	296

Nearest failing is the nearest measured failing probe rate. TBD marks workloads awaiting the new benchmark run. Rows issue and read-supply carry the run-20261014-01 values of mint and read-total-supply until then.

Read p50 latency is 6 to 7 ms, and read p95 latency is 7 to 10 ms (Table IX). Write p50 latency ranges from 788 to 2147 ms, and write p95 latency ranges from 2005 to 2195 ms. The eight write workloads confirmed below 0.5 ops/s have p50 values between 2060 and 2147 ms, close to the two-second batch timeout. The two highest-rate writes, create-pending and open-wallet, have p50 values of 1159 and 1161 ms.

VII. LIMITATIONS AND FUTURE WORK

The evaluation uses a single host with one peer per organization and no geographic distribution or network latency. Its Caliper results therefore apply only to the reported configuration and do not estimate national-scale performance. Each compliance workload is confirmed with one worker over three 30-second rounds. A distributed deployment and a graded-skew contention workload would be needed to characterize other operating conditions.

The shared Fabric channel does not provide transaction privacy, and the prototype does not use private data collections or privacy-preserving cryptography. The single orderer provides ordering but does not tolerate its own failure. The prototype also omits offline payment, the RTGS giro account that backs issuance, and any reverse path from wallet balances to a Distributor reserve. Wholesale and retail banks

TABLE IX
LATENCY PER WORKLOAD AT ITS CONFIRMED RATE (MEDIAN OF THREE ROUNDS)

Workload	p50 (ms)	p95 (ms)
rupiah-monetary		
register-distributor	802	2080
issue	2069	2085
interbank-transfer	TBD	TBD
redeem-reserve	2066	2122
read-supply	6	7
read-reserve	6	7
rupiah-controls		
create-pending	1159	2005
protect-subject	852	2070
create-staff-pending	2069	2078
protect-staff	796	2070
set-staff-role	792	2068
record-review	852	2071
approve-standard	874	2083
decide-high-risk	2060	2062
set-limits	804	2078
set-business-authority	793	2071
freeze-subject	788	2067
request-unfreeze	798	2076
approve-unfreeze	2060	2071
read-caller-identity	6	7
can-pay	6	7
read-limits	6	7
can-act-for	6	7
validator-read-subject	6	7
rupiah-payments		
open-wallet	1161	2035
transfer-retail	939	2113
transfer-merchant	2101	2134
convert-to-retail	TBD	TBD
fund-wallet	TBD	TBD
read-balance	7	10
read-receipt	6	8

Rows issue and read-supply carry the run-20261014-01 values of mint and read-total-supply. TBD marks workloads awaiting the new benchmark run.

share one DistributorMSP, so bank ownership relies on the `distributor_id` attribute.

The authorization, validation, and action-outcome checks cover the tested paths only. They do not establish formal security, production readiness, or cross-machine reproducibility. Future work should repeat the protocol on distributed hosts, instrument endorsement and commit phases, expand the contention workload, and evaluate privacy, offline payment, and interoperability.

The compliance study is limited to synthetic chaincode integration scenarios. It does not use live identity verification or screening feeds, submit reports to PPATK, or establish the effectiveness of a risk model or institutional compliance. Its backend HTTP path is exercised by functional tests but has not been load-measured. Performance results for the three chaincodes are limited to one Caliper worker on one host with 30-second confirmation windows. The run was interrupted by a host or power interruption and resumed under the same stamp from its last checkpoint. After the restart, the network was brought up again with existing identity material, and chaincode definitions already committed to the channel were skipped. The two payments reads measured after the restart (272 ops/s) are lower than the eight controls and monetary reads measured before it (296 ops/s). The paper does not attribute this difference to the restart, and it reports no fresh-network comparison for the payments reads. The failure classifier used in this run counted transport errors as transaction failures; the fix was made after the run and is not reflected in its results. The current harness stops a search on a transport error as inconclusive, without automatic retry.

VIII. CONCLUSION

This paper reports a two-tier Digital Rupiah prototype on a four-organization Hyperledger Fabric network. Its three chaincodes implement hierarchical issuance and distribution, retail transfers, organization-aware authorization, and read-only Observer access to subject records. The transfer protocol combines explicit eligibility, limit, and balance checks with Fabric MVCC.

The compliance study is the main study of this paper. As reported for run-20261014-01 under the earlier identifiers, its synthetic integration scenarios passed for distinct senior approval, restriction evidence, reserve funding, retail payment, read-only ValidatorMSP access, and expected endorsement failures; the post-rename correctness re-run is pending. It also reports throughput and latency for the same three chaincodes on a single host. This functional evidence does not depend on the Caliper results in Section VI.

ARTIFACT AVAILABILITY

The bi-coin repository at https://github.com/mfachrizalg/bi-coin contains the three chaincodes (`bi-coin-fabric/chaincode/monetary`, `controls`, and `payments`), the four-organization network definition (`bi-coin-fabric/network`), the Go backend, and the Caliper workloads (`bi-coin-fabric/benchmark/compliance`).
The results of compliance-run-20261014-01 are in `bi-coin-fabric/benchmark/results/compliance-run-20`
and Section VI reports them.

REFERENCES

[1] J. Kiff, J. Alwazir, S. Davidovic, A. Farias, A. Khan, T. Khiaonarong, M. Malaika, H. Monroe, N. Sugimoto, H. Tourpe, and P. X. Zhou, "A survey of research on retail central bank digital currency," *IMF Working Papers*, 2020.

[2] Bank Indonesia, "Proyek garuda: Wholesale digital rupiah cash ledger — proof of concept report," Bank Indonesia, Jakarta, Tech. Rep., 2024.

[3] ——, "Proyek garuda: Menavigasi arsitektur digital rupiah — white paper central bank digital currency," Bank Indonesia, Jakarta, Tech. Rep., 2022.

[4] R. A. Auer and R. Böhme, "The technology of retail central bank digital currency," *BIS Quarterly Review*, pp. 85–100, Mar. 2020. [Online]. Available: https://www.bis.org/publ/qtrpdf/r_qt2003j.htm

[5] Bhawana and S. Kumar, "Permission blockchain network based central bank digital currency," in *2021 IEEE 4th International Conference on Computing, Power and Communication Technologies (GUCON)*. IEEE, 2021, pp. 1–6.

[6] M. M. Islam and H. P. In, "A privacy-preserving transparent central bank digital currency system based on consortium blockchain and unspent transaction outputs," *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2372–2386, 2023.

[7] J. Hans, S. Khan, and D. Svetinovic, "Blockchain cbdc security threats using stride," in *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2023, pp. 522–529.

[8] J. Lovejoy, C. Fields, M. Virza, T. Frederick, D. Urness, K. Karwaski, A. Brownworth, and N. Narula, "A high performance payment processing system designed for central bank digital currencies," *IACR Cryptology ePrint Archive*, vol. 2022, p. 163, 2022. [Online]. Available: https://eprint.iacr.org/2022/163

[9] J. Ree, "Nigeria's enaira, one year after," International Monetary Fund, IMF Working Paper WP/23/104, 2023.

[10] People's Bank of China, "Progress of research and development of e-cny in china," People's Bank of China, Tech. Rep., 2021. [Online]. Available: https://www.pbc.gov.cn/en/3688110/3688172/4157443/4293696/2021072014364791207.pdf

[11] Central Bank of The Bahamas, "Project sand dollar: A digital fiat currency for the bahamas," Central Bank of The Bahamas, Nassau, Tech. Rep., 2020. [Online]. Available: https://www.centralbankbahamas.com/viewPDF/documents/2019-12-25-02-18-11-Project-Sanddollar.pdf

[12] V. Sethaput and S. Innet, "Sok: Blockchain applications in central bank digital currencies (cbdcs)," in *2025 7th International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2025, pp. 195–202.

[13] J. N. Aryani, "Designing central bank digital currency in indonesia: A critical literature review," 2023, tugas Akhir Sarjana, Sekolah Bisnis dan Manajemen, Institut Teknologi Bandung.

[14] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*. Porto, Portugal: ACM, 2018, pp. 1–15.

[15] Hyperledger Foundation, "Hyperledger caliper: A blockchain performance benchmark framework," 2024, accessed: Jun. 27, 2026. [Online]. Available: https://hyperledger.github.io/caliper/

[16] L. I. Hanif, "Rancang bangun sistem blockchain aplikasi meetcoin menggunakan kerangka kerja hyperledger fabric," Universitas Gadjah Mada, Yogyakarta, Skripsi Sarjana, Departemen Teknik Elektro dan Teknologi Informasi, 2024.

[17] Hyperledger Performance and Scale Working Group, "Hyperledger blockchain performance metrics," Hyperledger Foundation, Tech. Rep., 2018.

[18] A. Ahuja, S. Pani, S. Kanchanapalli, V. R, R. M. A, and S. Lodha, "Limbocoin: On the denial-of-service of token based retail cbdcs," in *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 2024.

[19] Bank for International Settlements, "Project aurum: A prototype for two-tier central bank digital currency (cbdc)," BIS Innovation Hub Hong Kong Centre, Hong Kong Monetary Authority and Hong Kong Applied Science and Technology Research Institute, Tech. Rep., Oct. 2022. [Online]. Available: https://www.bis.org/publ/othp57.pdf